

MODULE 07 · DOMAIN 2 - IMPLEMENTATION AND INTEGRATION

Agents That Survive Production

Task 2.1. AgentCore, Agents Classic, MCP, tool design, safeguards and multi-agent patterns.

DOMAIN 2 of 5 EXAM WEIGHT 26% MODULE 7 of 22

An agent is not a clever prompt. It is a **loop** in which a model decides what to do next, and that loop is what makes agents powerful and dangerous in equal measure.

The agent loop, one turn at a time

01

Goal

A task arrives, plus the tools available.

02

Reason

The model picks a tool and its arguments.

03

Act

Your code runs the tool and captures the result.

04

Observe

The result is appended to the conversation.

05

Repeat

Until the model answers, or a limit stops it.

ON THE EXAM

"Until a limit stops it" is doing enormous work. Without an explicit ceiling on iterations, time or spend, a confused agent will loop until something else breaks.

Do you actually need an agent?

- ◆ **Agents are slow.** Five tool calls means five model round trips, not one.
- ◆ **Agents are expensive.** Every turn resends the whole conversation, so cost grows quadratically.
- ◆ **Agents are non-deterministic.** The same request can take a different path on Tuesday.
- ◆ Use one when **the sequence of steps is genuinely unknown until runtime.**
- ◆ If you can draw the flowchart, build the flowchart. That is a workflow, and it will be cheaper, faster and testable.

ON THE EXAM

This is the highest-value judgement in the domain. A large share of production agents are workflows that someone made non-deterministic on purpose.

Agent or workflow?

AGENT

The model chooses the path

- Steps unknown until runtime
- Open-ended investigation or research
- Tool choice depends on what it finds
- Non-deterministic, and that is the point

WORKFLOW, STEP FUNCTIONS

You choose the path

- Steps known and repeatable
- Auditable, testable, replayable
- Retries, catch, parallel, callbacks
- Cheaper and dramatically faster

THE TIEBREAKER

The hybrid is usually right: a Step Functions workflow for the known structure, calling a bounded agent only for the step that genuinely needs judgement.

A payments company processes refund requests. Every request follows the same path: validate the order exists, check it falls inside the refund window, confirm no chargeback is open, calculate the refund amount from a published fee table, and issue the refund.

An engineer has built this as a Bedrock agent with five tools, arguing that the agent can handle edge cases flexibly. Finance has queried the cost, and the compliance team has asked for evidence that every refund followed the correct sequence of checks.

How should this be rebuilt?

- ☐ A Keep the agent but add stronger instructions specifying the required order of checks
- ☐ B Rebuild as an AWS Step Functions state machine, calling a foundation model only where judgement is required
- ☐ C Keep the agent and enable AgentCore Observability to produce the compliance evidence
- ☐ D Split into five agents, one per check, coordinated by a supervisor agent

◆ PAUSE AND DECIDE

How should this be rebuilt?

- A

 Stronger instructions on ordering
Instructions influence a probabilistic loop. Compliance asked for evidence every refund followed the sequence, and influence is not a guarantee.
- B

 Step Functions, model only where judgement is needed
The sequence is fixed and published, so the state machine enforces it structurally. Execution history is the compliance evidence, and it costs a fraction as much.
- C

 Agent plus AgentCore Observability
Records what the agent did, which proves the order varied rather than proving it did not. It also does nothing about cost.
- D

 Five agents and a supervisor
Multiplies every problem. More model calls, more cost, more non-determinism, and now a coordination layer that can also go wrong.

THE SIGNAL

"Every request follows the same path" is the sentence. A known, fixed sequence is a workflow. Building it as an agent buys flexibility nobody asked for and gives up the guarantee somebody did.

Three layers, and they are independent

- ◆ **Strands Agents** is an open-source SDK. The agent loop, tools and multi-agent patterns, in your own code.
- ◆ **Amazon Bedrock AgentCore** is the managed runtime. Where the agent runs, remembers, authenticates and is observed.
- ◆ **Kiro** is the agentic IDE. It is a development tool, not part of the runtime.
- ◆ AgentCore is **framework-agnostic and model-agnostic**. Strands, LangChain, or your own loop all run on it.
- ◆ You can use any layer without the others. That independence is the design intent.

ON THE EXAM

AgentCore is infrastructure, not a turnkey agent service. That distinction is why it is not a drop-in replacement for Bedrock Agents Classic, which was turnkey.

The AgentCore components

COMPONENT	WHAT IT SOLVES
Runtime	Somewhere secure to run, with session isolation
Memory	Short-term and long-term recall across sessions
Gateway	Turns APIs, Lambda functions and MCP servers into tools
Identity	Inbound auth, and outbound credentials to act on a user's behalf
Code Interpreter	A sandbox where the agent can execute code
Browser	A managed browser the agent can drive
Observability	Traces of every step, on CloudWatch and OpenTelemetry

ON THE EXAM

These are composable and independently usable. You can adopt Gateway alone in front of an existing agent, or Memory alone behind a Lambda-hosted loop.

Runtime and Memory

- ◆ **Runtime** supports execution windows up to **eight hours** with complete session isolation per user.
- ◆ That eight-hour window is the answer to Lambda's fifteen-minute ceiling for long-running agents.
- ◆ It supports the **A2A** agent-to-agent protocol, plus VPC, PrivateLink, CloudFormation and tagging.
- ◆ **Memory** offers short-term session context and long-term strategies: semantic, user preference and summary.
- ◆ Memory now supports **fine-grained access control**, using Cedar policies behind an OAuth-configured Gateway.

ON THE EXAM

Memory is the managed answer to module 3's point that conversation state is your problem. You either build it in DynamoDB yourself, or you use this.

Harness or Runtime?

AGENTCORE HARNESS

Config-driven

- Declare model, tools and instructions
- AgentCore runs the loop for you
- GA June 2026, no separate charge
- The closest analogue to Agents Classic

AGENTCORE RUNTIME

Your code, hosted

- You write the loop, in Strands or anything
- Full control of orchestration logic
- Any framework, any model, any protocol
- More power, more to maintain

THE TIEBREAKER

A team migrating from Agents Classic and wanting the managed feel should look at Harness. A team that already has a Strands agent wants Runtime.

Amazon Bedrock Agents Classic, by its parts

- ◆ **Action groups** define what the agent can do, described by an OpenAPI schema or a simplified function schema.
- ◆ **Lambda fulfilment**: the agent calls your function with the predicted action and parameters.
- ◆ **Return of control**: instead of calling Lambda, the agent hands the predicted action back to *your* application to execute.
- ◆ **Session attributes** and **prompt session attributes** carry context across turns.
- ◆ **Knowledge base association** and **multi-agent collaboration** with a supervisor and sub-agents.

ON THE EXAM

Return of control is the one people forget, and it is genuinely useful: it lets the agent decide while your existing system, with its own auth and audit, does the acting.

What maintenance mode actually means here

- ◆ Closed to new customers from **30 July 2026**. Existing agents keep working, with **no announced end of life**.
- ◆ The **model catalogue is frozen**. Models released after that date reach AgentCore, not Classic.
- ◆ The rename is **cosmetic at the API level**. The bedrock-agent namespace, SDK clients and IaC types are unchanged.
- ◆ Bedrock itself is untouched. Model inference, Knowledge Bases and Guardrails keep receiving new models.
- ◆ **On the exam, Classic is still a correct answer**. In your job, new builds go to AgentCore.

ON THE EXAM

The frozen model catalogue is the real cost, and it is easy to miss. A Classic agent quietly cannot use next year's models while everything around it can.

A company has an operational Bedrock Agents Classic agent handling internal IT requests. It works well, has been stable for eighteen months, and is covered by existing CloudFormation templates.

The team is now planning a second, unrelated agent for finance approvals, which must run investigations lasting up to two hours and requires per-user credential handling when calling third-party systems. An architect has proposed building the new agent on Agents Classic for consistency with the existing one.

What should the team do?

- ☐ A Build the new agent on Agents Classic for consistency, and migrate both later
- ☐ B Build the new agent on Amazon Bedrock AgentCore, and leave the existing Classic agent in place
- ☐ C Migrate the existing agent to AgentCore first, then build the new one there
- ☐ D Build the new agent as a Step Functions workflow to avoid the platform question entirely

◆ PAUSE AND DECIDE

What should the team do?

A Build on Classic for consistency

Classic is closed to new customers, so this may not even be possible. Two-hour runs and per-user outbound credentials are also outside what it does well.

C Migrate the existing agent first

Blocks a needed capability behind a migration nobody requires. AWS states existing Classic agents continue normally with no end-of-life date.

B AgentCore for the new agent, leave Classic alone

Runtime handles multi-hour execution and Identity handles outbound credentials. The working agent has no forced deadline, so migrating it is unfunded work.

D Step Functions instead

Investigations of unknown shape lasting two hours are the agent case. This trades the right tool for platform avoidance.

THE SIGNAL

Two requirements name their answers: "up to two hours" is AgentCore Runtime's eight-hour window, and "per-user credentials to third parties" is AgentCore Identity outbound auth.

A tool is a **contract**, not a function. The model never sees your code. It sees a name, a description and a parameter schema, and it decides from those alone.

Model Context Protocol

- ◆ **An open standard** for connecting models to tools. One protocol instead of a bespoke integration per framework.
- ◆ A **server** exposes tools. A **client**, inside your agent, discovers and calls them.
- ◆ Write a tool once as an MCP server, and every MCP-aware agent in the organisation can use it.
- ◆ **AgentCore Gateway** speaks MCP, and can turn existing APIs and Lambda functions into MCP tools.
- ◆ Gateway supports **OAuth and IAM** authorisation, so tool access is governed rather than open.

ON THE EXAM

The exam guide names MCP in three separate skills. Treat it as the assumed default for agent-tool integration rather than an advanced topic.

Host an MCP server on Lambda or ECS?

AWS LAMBDA

Stateless, lightweight tools

- Each call is independent
- Scales to zero, pay per invocation
- Fifteen-minute ceiling per call
- Cold starts on infrequent tools

AMAZON ECS

Complex or stateful tools

- Long-running or persistent connections
- Heavy dependencies and large images
- State held between calls
- Always on, so always billed

THE TIEBREAKER

Skill 2.1.7 names both explicitly: Lambda for stateless MCP servers providing lightweight tool access, ECS for MCP servers providing complex tools. That phrasing is close to a quotation.

What belongs in every tool implementation

- ◆ **Validate parameters.** The model generated them, so treat them as untrusted input every time.
- ◆ **Return errors as data**, describing what went wrong. The agent can then correct itself and retry.
- ◆ Throwing an unhandled exception gives the model nothing to reason about, so it usually repeats the same call.
- ◆ **Keep responses small.** Every tool result becomes context the agent pays for on every subsequent turn.
- ◆ **Make tools idempotent** where you can. Agents retry, and retries of a payment are not a theoretical concern.

ON THE EXAM

"Return errors as data" is the counterintuitive one. A tool that replies "no customer with that ID; try searching by email" turns a failure into the agent's next correct step.

A retailer has four separate agent applications, each with its own hand-written integration to the same inventory API. When the API added a field, three of the four broke, and a fourth silently returned stale data for a week.

The platform team wants one governed integration that every current and future agent can use, with access controlled per agent and no code change required in the agents when the API evolves. Some tools will run long database queries taking several minutes.

What should the platform team build?

- ☐ A A shared Python library that each agent imports, versioned in CodeArtifact
- ☐ B An MCP server exposing the inventory tools, fronted by AgentCore Gateway with per-agent authorisation
- ☐ C An Amazon API Gateway REST API in front of the inventory system, called directly by each agent
- ☐ D One shared Bedrock agent that the other four agents call as a sub-agent

◆ PAUSE AND DECIDE

What should the platform team build?

- A

Shared Python library

Centralises the code and not the deployment. Each agent must upgrade its dependency, so the API change still reaches four teams.
- B

MCP server behind AgentCore Gateway

One tool definition, discovered over a standard protocol, with per-agent authorisation at the Gateway. Schema changes propagate without touching any agent.
- C

REST API called directly

Solves the plumbing and not the contract. Each agent still hand-writes a tool definition, which is precisely what broke.
- D

A shared agent as a sub-agent

Puts a probabilistic model in front of a deterministic lookup. Four times the cost and latency for an inventory query.

THE SIGNAL

"No code change in the agents when the API evolves" rules out anything the agents import or hand-write. Only a discovered, protocol-level contract meets that.

Bounding the blast radius

- ◆ **Stopping conditions.** A maximum iteration count. Without one, a confused agent loops until something else fails.
- ◆ **Timeouts.** Wall-clock limits on the whole session and on each individual tool call.
- ◆ **IAM as the real boundary.** The agent's execution role is what it *can* do; the prompt is only what it *should* do.
- ◆ **Circuit breakers.** Stop calling a failing dependency rather than retrying it thirty times.
- ◆ **Human approval before irreversible actions.** Refunds, deletions, emails to customers, anything with a real-world effect.

ON THE EXAM

Prompt instructions are guidance, not enforcement. If an agent must never delete a record, the deny must be in the IAM policy, because the instruction can be talked around.

A customer service agent has access to a refund tool, an order lookup tool and an email tool. Its instructions state that refunds above £500 must be escalated to a human and never issued automatically.

During a load test, a crafted customer message persuaded the agent that it was operating in a test environment where limits did not apply, and it issued a £2,400 refund. In a separate incident an agent looped 340 times calling order lookup before the session was killed manually.

Which combination addresses both incidents?

- | | |
|--|--|
| A Strengthen the system prompt to state the £500 limit more forcefully and forbid test-environment claims | B Enforce the limit in the refund tool itself and in its IAM permissions, and configure a maximum iteration count and session timeout |
| C Apply a Bedrock guardrail with a denied topic covering refund limits | D Route all refunds through Amazon A2I for human review |

◆ PAUSE AND DECIDE

Which combination addresses both incidents?

- A** Strengthen the prompt

The prompt already stated the limit and was argued around. A more forceful version of a control that just failed is not a control.
- B** Enforce in tool and IAM, plus iteration cap and timeout

The tool cannot exceed the limit whatever the model believes, and the iteration cap bounds the loop. One fix per incident.
- C** Guardrail denied topic

Guardrails filter content in prompts and responses. They do not inspect a tool call's parameters or stop a function executing.
- D** A2I for all refunds

Addresses the refund incident by removing automation entirely, and does nothing about the 340-iteration loop.

THE SIGNAL

Two incidents, so a correct answer must fix both. The refund was a control-placement failure; the loop was a missing stopping condition. Only B covers each.

Putting a human in the loop

- ◆ **Step Functions task tokens** pause a workflow for hours or days awaiting an approval callback.
- ◆ **Amazon Augmented AI** provides managed human review workflows, though it entered maintenance in 2026.
- ◆ **API Gateway** collects structured feedback and approvals from your own interface.
- ◆ Gate on the **action**, not on the confidence score. Irreversible actions get review regardless of how sure the model was.
- ◆ Captured approvals and rejections become **evaluation data**, which module 16 uses directly.

ON THE EXAM

A2I is still a correct exam answer for human review workflows and entered maintenance mode on 30 June 2026. Teach the answer, know the reality.

Ways to coordinate several agents

OPTION	SHAPE	REACH FOR IT WHEN
Strands multi-agent	Your own code, open source	You already build with Strands
AWS Agent Squad	Orchestrator with a classifier	Routing heterogeneous agents
Classic collaboration	Managed supervisor, sub-agents	Existing Agents Classic estate
Step Functions	Explicit orchestration	Coordination must be deterministic

ON THE EXAM

Agent Squad uses an LLM classifier to route each request to the best-suited agent, scoping state by user, session and agent. That routing role is what distinguishes it.

Multi-agent, and when not to

- ◆ **Supervisor and specialists.** A router sends each request to a domain agent. The commonest useful pattern.
- ◆ **Parallel then aggregate.** Several models attempt the task, and custom logic combines the results.
- ◆ **Every additional agent multiplies cost, latency and failure modes.** Coordination is not free.
- ◆ Split when agents need **genuinely different tools, permissions or models**, not when a prompt is getting long.
- ◆ A single agent with well-described tools beats three agents with vague ones, most of the time.

ON THE EXAM

Skill 2.1.4 is about coordinating specialised models, and its cheapest form is not multi-agent at all: route simple requests to a small model and hard ones to a large one.

A startup's support agent handles billing, technical and account questions with one model and eleven tools. It answers correctly about 80% of the time. The commonest failure is picking the wrong tool, particularly between "lookup_invoice" and "lookup_subscription", whose descriptions are two words each.

The team has proposed splitting into three specialist agents behind a supervisor, expecting each to reason better over a smaller tool set. They have not yet measured which tool confusions cause which failures.

What should the team do first?

- ☐ A Build the three-agent supervisor architecture as proposed
- ☐ B Instrument tool selection to identify the failing pairs, then rewrite the tool descriptions and parameter schemas
- ☐ C Switch to a larger foundation model with stronger reasoning
- ☐ D Add a guardrail to block responses when the agent is uncertain

◆ PAUSE AND DECIDE

What should the team do first?

- A

 Build the supervisor architecture
Adds a routing model, three loops and coordination cost, while carrying the two-word descriptions into whichever agent inherits both tools.
- C

 Larger model
A stronger model still cannot distinguish two tools described in two words each. It pays more per call for the same ambiguity.
- B

 Instrument, then rewrite the tool contracts
The stated failure is tool selection, and selection happens from the description alone. Costs days, and the diagnosis survives whatever architecture follows.
- D

 Guardrail on uncertainty
Guardrails filter content categories. They have no view of tool selection confidence, and blocking answers does not make them correct.

THE SIGNAL

"Descriptions are two words each" is the fault, stated plainly. And "they have not yet measured" is the exam telling you that the proposed architecture change is a guess.

The five things to carry forward

- ◆ **If you can draw the flowchart, build the flowchart.** Agents are for genuinely unknown paths.
- ◆ **AgentCore** is the runtime: Runtime, Memory, Gateway, Identity, Code Interpreter, Browser, Observability.
- ◆ **Agents Classic** is still examinable. Action groups, return of control, session attributes, frozen model catalogue.
- ◆ **A tool is a contract.** Wrong tool chosen means a bad description, not bad code. Return errors as data.
- ◆ **Controls belong in IAM and in the tool**, never only in the prompt. Cap iterations, time and spend.

ON THE EXAM

Module 8 stays in Domain 2 and moves to the API layer: streaming, async, throttling, backoff and intelligent routing.